UNIVERSIDADE FEDERAL DA GRANDE DOURADOS

Faculdade de Ciências Exatas e Tecnologia
Disciplina de Sistemas Embarcados

Painel embarcado para telemetria e navegação no BeamNG drive

Professor Thayuan Sousa

1. Apresentação do projeto

O projeto implementa um visor físico para o simulador BeamNG.drive, utilizando ESP32 e display touch de 2,4 polegadas. A interface possui duas páginas: uma apresenta os instrumentos do veículo e a outra exibe um minimapa gerado com os dados internos de navegação do jogo. O toque e o arrasto permitem alternar entre essas páginas.

Necessidade e justificativa

Durante a condução em simuladores, a consulta aos instrumentos e à navegação pode ocupar parte da área de visão do jogo. A necessidade escolhida foi reunir essas informações em um dispositivo físico compacto, instalado próximo ao jogador, com interação direta pelo toque e apresentação adequada a uma tela de pequena resolução.

A solução integra entrada física, processamento concorrente, comunicação com o computador e saída gráfica em um único protótipo. O simulador fornece os dados do veículo; o ESP32 recebe as atualizações, trata os quadros e controla a interface local.

Objetivos

O objetivo geral é desenvolver um painel embarcado interativo para acompanhar a condução e a rota no BeamNG.drive. Os objetivos específicos são organizar o firmware com FreeRTOS, receber telemetria por USB, interpretar gestos no touch, apresentar alertas e exibir um minimapa independente da captura do monitor.

Visão geral do sistema

BeamNG.drive e mod Lua → UDP local → programa Python → USB serial → ESP32 → display e touch.

A versão final utiliza navegação nativa do jogo. A captura de tela foi explorada durante o desenvolvimento, mas não é necessária ao funcionamento dessa versão.

2. Protótipo físico e conexões

O protótipo utiliza uma placa ESP32, display TFT ILI9341 de 320 × 240 pixels com touch resistivo SPI compatível com XPT2046, cabo USB e conexões entre os módulos. A USB alimenta a placa e transporta os dados do computador. O firmware utiliza a orientação de paisagem configurada por setRotation(3).

Sensor de toque e saída gráfica

O painel touch constitui a entrada sensorial física: o controlador fornece coordenadas e uma leitura bruta associada ao contato. Após a calibração, o ESP32 converte essas leituras para pixels e identifica o gesto realizado. As grandezas do veículo são dados da simulação, e não medições de sensores automotivos instalados no protótipo.

Ligação	Sinal utilizado	ESP32
SPI compartilhado	Tela_SCK e Touch_CLK	GPIO 18
SPI compartilhado	Tela MOSI/SDI e Touch_DIN	GPIO 23
SPI compartilhado	Tela MISO/SDO e Touch_DO	GPIO 19
Display	CS	GPIO 5
Display	DC	GPIO 2
Display	RESET	GPIO 4
Touch	T_CS	GPIO 22
Touch	T_IRQ	- 1 (Não conectado)
Referência elétrica	GND comum	GND

Display e touch compartilham relógio e linhas de dados, com pinos CS separados. O touch é consultado por software, com IRQ configurado como −1.

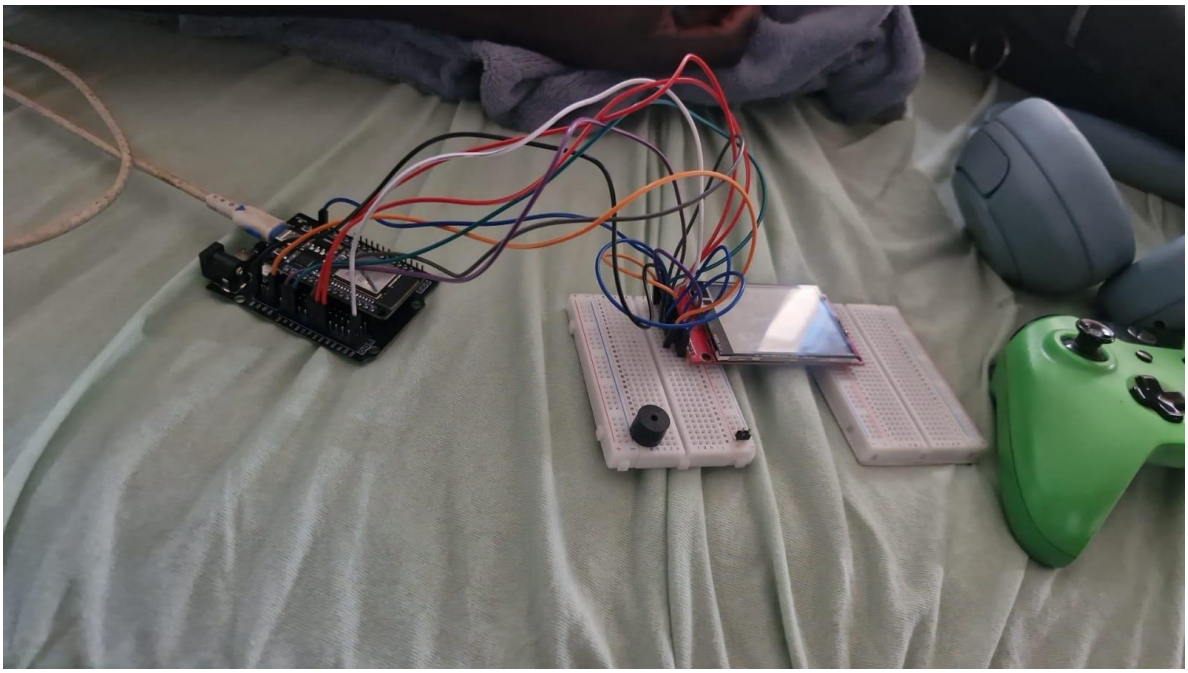


Figura 1. Montagem do ESP32 com o display touch e a conexão USB

3 Organização do software e multitarefa

O firmware foi organizado em duas tarefas concorrentes. A recepção é criada por xTaskCreate(), e a interface executa na loopTask disponibilizada pelo ambiente Arduino sobre FreeRTOS [3]. As prioridades relativas adotadas são apresentadas a seguir.

Tarefa	Prioridade	Responsabilidade	Justificativa
Recepção serial SerialRX	2	Solicitar a página ao PC, receber o quadro, conferir tamanho e CRC32 e disponibilizar o conteúdo à interface.	Atender a comunicação com precedência e reduzir o risco de acúmulo de dados na entrada serial. A tarefa cede processamento durante a espera por bytes.
Interface e touch loopTask	1	Consultar o touch, reconhecer gestos, desenhar instrumentos, decodificar o JPEG do mapa e atualizar o aviso SHIFT.	Reunir os acessos ao barramento SPI em uma única tarefa e manter a atualização visual cooperativa com a recepção.

Comunicação entre tarefas

A fila de páginas contém a seleção mais recente da interface. A tarefa serial consulta essa fila antes de solicitar um novo quadro ao computador. Outra fila transporta o descritor do quadro recebido, incluindo tipo, tamanho e página de destino.

Um semáforo binário controla o tempo de vida do buffer compartilhado: após entregar um quadro, a tarefa serial aguarda a interface terminar sua leitura antes de reutilizar a memória. Como display e touch são operados somente pela tarefa de interface, não há acessos concorrentes ao SPI entre essas tarefas.

Temporização e tratamento de erros

O firmware não utiliza delay(). Os intervalos cooperativos são implementados com vTaskDelay(), e millis() é usado na leitura de gestos, no piscar do SHIFT e na verificação de ausência de quadros. A leitura serial possui limite de espera de 3 segundos; após 5 segundos sem quadro válido, a interface apresenta uma mensagem de falta de resposta do computador.

Os quadros são limitados a 50 000 bytes e protegidos por CRC32. O conteúdo numérico é validado antes da exibição, e quadros destinados a uma página anterior são descartados após a troca de tela. Falhas na alocação de memória ou na criação das estruturas de comunicação são sinalizadas durante a inicialização.

Caminhos de comunicação com o jogo

Os instrumentos utilizam OutGauge via UDP local na porta 4444. A navegação utiliza o mod Lua Visor Rally Nativo, que envia posição relativa, direção, estradas próximas e pontos da rota em mensagens RLYNAV1 na porta 4445. O envio de navegação é configurado para cinco atualizações por segundo. Esses caminhos seguem a possibilidade de integração por UDP e protocolos próprios do BeamNG.

O programa Python converte os valores, desenha o mapa e transmite quadros RLY1 pela USB a 460800 baud. O cabeçalho possui tipo de conteúdo, tamanho e CRC32. O ESP32 solicita um quadro por vez, conforme a página selecionada. A rede UDP fica no próprio computador; o enlace com o ESP32 é a USB.

4. Interface e navegação por toque

Página de instrumentos

A primeira página apresenta velocidade, RPM, marcha, combustível, pressão do turbo e indicador de ABS. O aviso SHIFT pisca em ciclos de 250 ms aceso e 250 ms apagado. Seu acionamento utiliza o sinal do jogo quando disponível, ou um limite de RPM configurável no programa Python. Em neutro e ré, o aviso é desativado.

O ícone ABS reproduz o sinal informado pelo jogo; ele não distingue, sozinho, atuação do sistema e desativação. A disponibilidade do turbo também depende do veículo. Dados indisponíveis são apresentados de forma distinta de uma leitura numérica válida.



Figura 2. Instrumentos do carro exibidos no display físico

Página de minimapa nativo

A segunda página apresenta estradas em cinza, rota em laranja e marcador do carro em azul. O desenho é calculado no computador a partir dos dados de navegação, com a frente do veículo orientada para cima. O ESP32 recebe e decodifica a imagem JPEG, sem capturar o monitor. O mapa ocupa 320 × 208 pixels, preservando o rodapé de navegação.

O arrasto para a esquerda abre o mapa, e o arrasto para a direita retorna aos instrumentos. Os botões inferiores oferecem uma alternativa por toque. A calibração converte as coordenadas brutas do sensor para a orientação física do display.

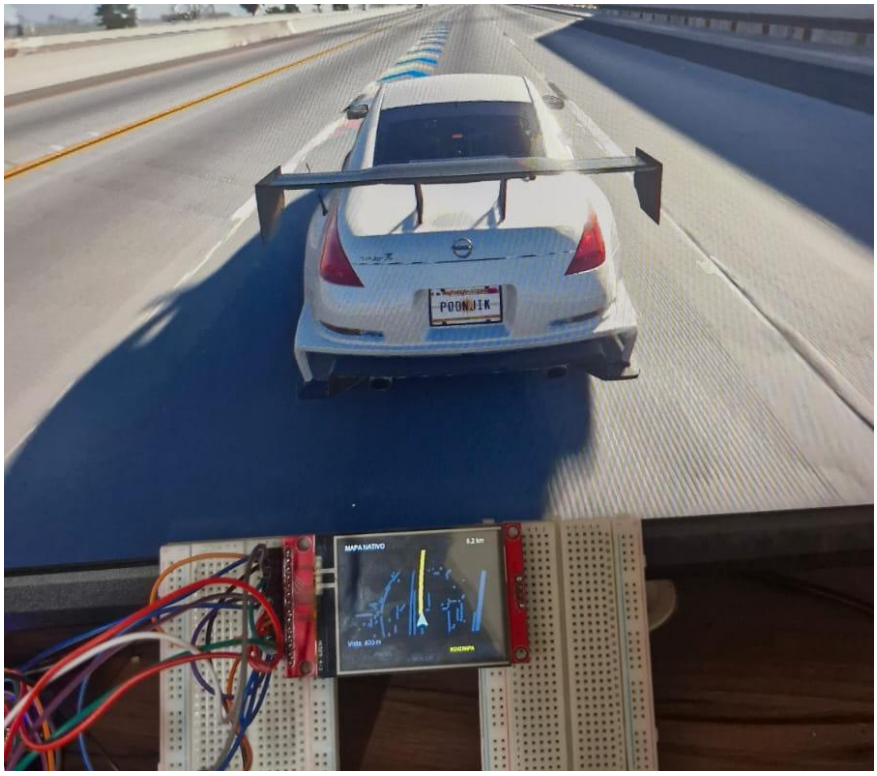


Figura 3. Rota nativa do BeamNG apresentada no visor

5 Validação e resultados

A validação ocorreu de forma incremental: primeiro foram verificadas a imagem do display e a leitura do touch; depois, a comunicação com o computador, os instrumentos e a navegação. Ao final, foi confirmado no protótipo o funcionamento das duas páginas, do arrasto e do minimapa nativo. A avaliação foi funcional e qualitativa, sem ensaio formal de latência ou taxa de quadros.

Verificação	Resultado obtido
Display e touch	Exibição em paisagem e coordenadas calibradas para a posição física da tela.
Integração com o computador	Inicialização da placa e troca de quadros pela COM6 confirmadas durante os testes.
Instrumentos e navegação	Painel, mudança de páginas e minimapa nativo confirmados em funcionamento pelo operador.
Dados e protocolo	Testes de software verificaram conversões, campos OutGauge, CRC32, tamanho de quadros e recepção UDP local.
Exportador Lua	Rotas e estradas exercitadas com uma API simulada em teste de software; o carregamento do mod foi observado no jogo.

Correções realizadas

A falta de memória na inicialização foi tratada reduzindo buffers e ajustando a criação do mostrador para depois da inicialização do ambiente Arduino. A sincronização entre recepção e desenho impede que um novo quadro sobrescreva a imagem ainda em uso. A seleção por captura do monitor foi substituída pela leitura de navegação em Lua na versão final.

Limitações e continuidade

O sistema depende do computador e do jogo em execução. O minimapa é esquemático, sem texturas de terreno, e utiliza a rota ativa do sistema core_groundMarkers. Sem destino definido, são mostradas as estradas locais. A cobertura depende do grafo de navegação do mapa; algumas missões e trilhas podem utilizar outros sistemas. O limite de SHIFT deve ser ajustado ao veículo quando o jogo não fornece esse sinal.

6 Conclusão

O projeto reuniu leitura de toque, interface gráfica, comunicação serial e execução concorrente com FreeRTOS em um painel aplicado à simulação de direção. A separação das responsabilidades entre computador e ESP32 permitiu apresentar instrumentos e navegação em um display compacto. A demonstração ao professor e as fotografias do protótipo complementam a validação exigida pela atividade.